

1) A company wants to develop three AI applications: generating product descriptions, creating realistic product images, and your knowledge of Autoregressive Models, GANs and VAEs to select an appropriate model for each application and justify your choice.

A. Generative AI relies on diverse mathematical and probabilistic frameworks to synthesize data across text, vision and latent representations. Selecting an appropriate architecture requires matching the data modality and operational objective to the inductive biases of the model.

1. Generating Product Descriptions: Autoregressive Models: for text generation, Autoregressive Models are optimal choice

Working: AR models factorize the joint probability of a sequence into a series of conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t | x_{1:t-1})$$

tokens are generated sequentially, conditioning each new word on the preceding content

Justification: Text is inherently discrete, sequential and context dependent, AR models maintain long-range syntactic and semantic dependencies via self-attention mechanisms

2. Creating Realistic Product Images: Generative Adversarial Networks (GANs):

For generating high fidelity marketing visuals, GANs are most suitable architecture

Working: GANs operate via a zero-sum game between a Generator (G) and Discriminator (D):

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

G - synthesizes image from latent noise.

D - evaluates authenticity driving G to produce visually.

Justification: Commercial imagery requires sharp textures and precise edge definitions. Likelihood-based models often optimize pixel-wise mean squared error (MSE), causing blurry artifacts.

3. Reconstructing Noisy images: Variational Encoders (VAEs):

For image denoising and structured latent extraction, VAEs provide the required framework.

Working: VAEs employ an encoder-decoder architecture that optimizes the Evidence Lower Bound (ELBO):

$$L_{\text{ELBO}}(\theta, \phi; x) = \mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) || p(z))$$

Justification: VAEs learn a continuous, regularized latent manifold. When given a corrupted input, the probabilistic bottleneck discards unstructured, high-frequency noise while retaining the underlying global semantics, allowing the decoder to reconstruct a smooth noise-free representation.

2. An autoregressive model assigns the probabilities $P(AI) = 0.8$, $P(\text{generated} | AI) = 0.6$, and $P(\text{text} | AI, \text{generated}) = 0.7$. Calculate the probability of generating the sequence "AI generated text".

A) given
 $P(AI) = 0.8$, $P(\text{generated} | AI) = 0.6$
 $P(\text{text} | AI, \text{generated}) = 0.7$

Work 2nd: 11.11

23BA1A61C6

Formula:

$$\prod_{i=1}^n p(w_i | w_1, w_2, \dots, w_{i-1})$$

$$p(w_1, w_2, w_3, \dots, w_n)$$

for 3-token sequence

$W = ("AI", "generates", "text")$

$$p("AI \text{ generates text"}) = p(AI) \times p(\text{generates} | AI)$$

$$\times p(\text{text} | AI, \text{generates})$$

$$= 0.8 \times 0.6 \times 0.7$$

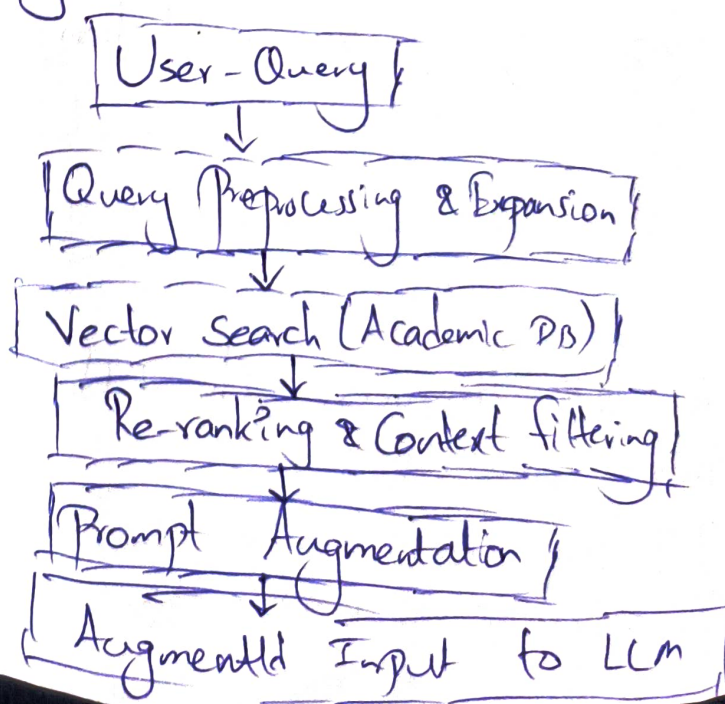
$$= 0.336$$

$$\therefore p("AI \text{ generates text"}) = 0.336$$

3. Apply the RAG process and show the main steps the system should follow before generative the answer for the user query. "What are the requirements for a student to be promoted to the next academic year?"

A. Retrieval Augmented Generation (RAG) optimizes the output of a Large Language Model (LLM) by referencing an authoritative external knowledge base before generating a response. For the given academic policy query, the pre-trained phase ensures that the model retrieves exact institution regulations rather than relying on parametric memory.

Pre-Generation Workflow:



Step-1: Query Processing & Expansion
 Input: A raw query "What are the requirements for a student to be promoted to the next academic year?"

- * Text cleaning, stopword removal and normalisation
- * Domain specific query expansion

Step-2: Dense Query Embedding Generation

- * The expanded textual query is passed into a trained Bi-encoder embedding model.
- * e_q = Dense numerical vector returned by embedding model.

Step-3: Similarity Search and Dense Retrieval:

- * The query vector e_q is matched against an indexed vector database storing chunked university bylaws and academic handbooks.
- * Retrieval metric: Cosine Similarity is computed between e_q and stored vectors e_d .

$$\text{Sim}(e_q, e_d) = \frac{e_q \cdot e_d}{\|e_q\| \|e_d\|}$$

- * System retrieves top-k document chunks.

Step-4: Reranking and Context Filtering

- * The retrieved chunks are processed by a cross-encoder reranker to compute fine-grained token-level semantic relevance
- * Irrelevant chunks strictly detailing promotion rules

Step-5: Prompt Augmentation

- * The verified context chunks are packaged with the original user query and a strict system instruction to form an augmented prompt payload

A 4. An LLM answers, "The Eiffel Tower is located in Rome." Find the problem and apply

A. 1. Problem Identification: Factual Hallucination.

* Nature of error = The LLM generates an incorrect fact due to flawed statistical associations with its parametric weights rather than retrieval.

23891AG166
* Underlying Cause: Next token probability optimization $P(w_t/w_{t-1})$ favors plausible sounding text over factual correctness, leading to geographical conflation.

Mitigation Techniques:

Technique - 1: Retrieval Augment Generation (RAG):

- * Mechanism: Queries an external verified knowledge base before generation.
- * Application:
 - Retrieve the ground-truth document chunk:
"The Eiffel Tower is located in Paris, France."
 - Inject the chunk into the prompt as reference context with the instruction: "Answer strictly based on the provided context."
- * Outcome: Replaces unreliable parametric recall with verified, non-parametric evidence.

Technique - 2: Decoding Control (Greedy Decoding / Low Temperature):

- * Mechanism: Minimizes sampling randomness by lowering the softmax temperature ($T \rightarrow 0$).

$$P(w_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- * Applications:

- Setting $T=0$ forces deterministic selection of the highest-probability canonical token.
- Combine with few-shot prompting to anchor

- * Outcome: Eliminates high-entropy token deviations and suppresses erroneous exploratory generation.

5. A student provides an image of a bar chart to a multimodal LLM and asks, "Which category has the highest value?" Demonstrate how multi-modal LLM would use the image and text input to produce the answer.

A- Architecture & Preprocessing Pipeline: A multimodal model processes heterogeneous visual and textual inputs. Core stages: Visual Feature Extraction, Cross-Modal Alignment, and Auto-regressive Text generation. large language through 3

Step-1: Visual Tokenisation & Encoding: It is divided into non-overlapping patches and passed through a vision transformer encoder.

- * The input bar chart image I is divided into non-overlapping patches and passed through a vision transformer encoder.
- * The encoder extracts spatial representations capturing visual primitives.

Step-2: Cross-Modal Projection & Alignment

- * A projection module maps the visual embeddings Z_v into the textual embedding space.
- * This aligns visual features with the word token dimension so the language backbone interprets image regions as visual tokens.

Step-3: Text Tokenisation & Prompt Classification.

- * The user query "Which category has the highest value?" is tokenised into text embeddings H_t .
- * The unified sequence is formed by prepending the visual tokens to text tokens.

Step-4: Cross-Attention & Visual Reasoning.

- * Within the transformer backbone, self-attention layers compute cross-modal attention between text prompt and visual tokens.
- * Query tokens attend directly to the highest bar height on y-axis and trace down to its corresponding label on x-axis.

Step-5: Autoregressive Answer Generation.

- * The language decoder generates the final prediction $\hat{y}$ by token based on joint attention over the chart features.
- * Output: Identifies the target category and output: "Category $[x]$ has the highest value."